

6장 서비스 1

6.1 서비스 소개

- 6.1.1 실습 환경 구성
- 6.1.2 서비스 등장 배경
- 6.1.3 서비스 동작
- 6.1.4 서비스 종류

6.2 ClusterIP 서비스

- 6.2.1 파드 생성
- 6.2.2 서비스 생성
- 6.2.3 iptables 규칙
- 6.2.4 셀렉터
- 6.2.5 세션어피니티

6.3 NodePort 서비스

- 6.3.1 파드 생성
- 6.3.2 서비스 생성
- 6.3.3 client 가상머신 접속 테스트
- 6.3.4 iptables 규칙
- 6.3.5 externalTrafficPolicy 설정

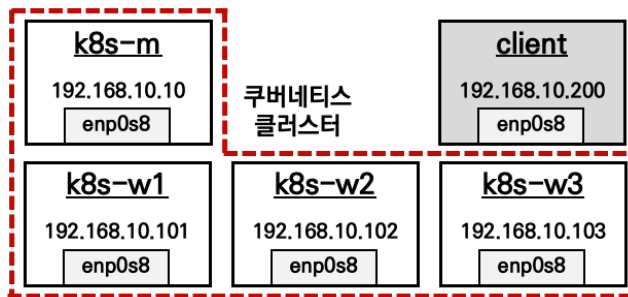
6.1 서비스 소개

서비스 <Service> 소개를 위한 실습 환경을 먼저 구성하겠습니다.

6.1.1 실습 환경 구성

CNI 는 칼리코 CNI 를 사용합니다.

[그림 6-1] 실습 환경 구성도



vagrant 로 5대의 가상머신을 배포하며, 쿠버네티스 클러스터에 마스터 노드 1대와, 워커 노드 3대(k8s-w1 k8s-w1 k8s-w3) 로 구성됩니다. client 이름의 가상머신은 서비스 접속 테스트 용도로 사용됩니다.

실습 환경을 배포하겠습니다.

```
터미널
# 베이그런트 스크립트 파일을 다운로드
curl -O https://raw.githubusercontent.com/gasida/book-k8s-network/main/6/Vagrantfile

# 배포 실행 : PC 환경에 따라 다르지만, 보통 20분~30분 정도 소요
vagrant up

# 배포된 가상머신 상태 확인 : 5대의 가상머신이 정상 동작 중
vagrant status

Current machine states:
k8s-m      running (virtualbox)
```

```
k8s-w1      running (virtualbox)
k8s-w2      running (virtualbox)
k8s-w3      running (virtualbox)
client      running (virtualbox)
```

SSH 접속 : 각각 가상머신 접속

```
vagrant ssh k8s-m
```

```
vagrant ssh k8s-w1
```

```
vagrant ssh k8s-w1
```

```
vagrant ssh k8s-w2
```

```
vagrant ssh client
```

5장에서 설치한 쿠버네티스 스크립트와 다른 부분들을 살펴보겠습니다.

 터미널

베이그런트 스크립트 파일 확인

```
cat Vagrantfile
```

...(생략)...

max number of worker nodes : Ex) N = 3 # 워커 노드 3대를 배포

N = 3

...(생략)...

#-----Client PC Subnet # client 가상머신을 배포

...(생략)...

초기화 설정을 실행하는 스크립트 파일 확인

```
cat init_cfg.sh
```

...(생략)...

```
echo "[TASK 8] Install containerd.io" # 컨테이너 런타임(CRI)에 containerd 를 설치
```

...(생략)...

마스터 노드를 설정하는 스크립트 파일 확인

```
cat master.sh
```

...(생략)...

```
echo "[TASK 1] Initial Kubernetes - Pod CIDR 172.16.0.0/16 , Service CIDR 10.200.1.0/24 , API Server 192.168.10.10"
```

```
kubeadm init --token 123456.1234567890123456 --token-ttl 0 --pod-network-cidr=172.16.0.0/16 --apiserver-advertise-
```

```
address=192.168.10.10 --service-cidr 10.200.1.0/24 # 서비스 사용 네트워크 대역을 10.200.1.0/24 로 지정
```

...(생략)...

설치 후 환경을 간단히 확인해보겠습니다.

터미널

SSH 접속 : 마스터 노드 접속

vagrant ssh k8s-m

노드 정보 확인 : 워커 노드 3대 확인

root@k8s-m:~# **kubectl get node**

NAME	STATUS	ROLES	AGE	VERSION
k8s-m	Ready	control-plane,master	114m	v1.23.6
k8s-w1	Ready	<none>	112m	v1.23.6
k8s-w2	Ready	<none>	110m	v1.23.6
k8s-w3	Ready	<none>	108m	v1.23.6

컨테이너 런타임 확인 : docker 대신 containerd 를 사용 중

kubectl get node -o=custom-columns=NAME:.metadata.name,CRI:.status.nodeInfo.containerRuntimeVersion

NAME	CRI
k8s-m	containerd://1.6.4
k8s-w1	containerd://1.6.4
k8s-w2	containerd://1.6.4
k8s-w3	containerd://1.6.4

파드와 서비스가 사용 할 수 있는 네트워크 대역 정보 확인

root@k8s-m:~# **kubectl cluster-info dump | grep -m 2 -E "cluster-cidr|service-cluster-ip-range"**

"--service-cluster-ip-range=10.200.1.0/24",

"--cluster-cidr=172.16.0.0/16",

칼리코 모드 정보 확인 : Direct 모드 사용 중

root@k8s-m:~# **calicoctl get ippool -owide**

NAME	CIDR	NAT	IPIPMODE	VXLANMODE	DISABLED	DISABLEBGPEXPORT	SELECTOR
default-ipv4-ippool	172.16.0.0/16	true	Never		false		all()

서비스와 엔드포인트 정보 확인 : 기본 설정 정보 확인

root@k8s-m:~# **kubectl get service,endpoints**

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.200.1.1	<none>	443/TCP	5m28s

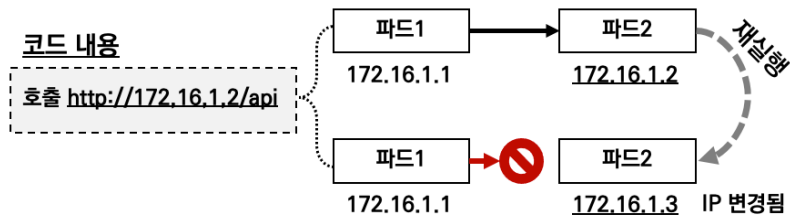
NAME	ENDPOINTS	AGE
endpoints/kubernetes	192.168.10.10:6443	5m28s

6.1.2 서비스 등장 배경

파드 간 통신 시 특정한 상황에서 통신에 문제가 발생하게 되고, 이를 해결하기 위해서 **서비스** <Service> 가 등장하게 되었습니다.

예를 들면 아래 그림과 같은 파드1 애플리케이션은 파드2의 172.16.1.2 IP 로 통신할 수 있게 코드가 작성되어 있습니다. 이 상태에서 그림 아래 부분 처럼 파드2가 재실행 된다면 파드2의 IP 가 변경되고, 파드1 애플리케이션은 여전히 172.16.1.2 로 통신을 시도하기 때문에 문제가 발생하게 됩니다.

[그림 6-2] 파드 간 통신 시 발생 가능한 문제



서비스는 파드에 대해서 **고정 IP** 와 **도메인네임** <Domain Name> 를 제공합니다.

파드1 애플리케이션은 서비스의 10.200.1.1 IP 로 통신 요청 시, 서비스는 파드2의 IP인 172.16.1.2 로 전달하게 됩니다.

만약 파드2 가 재실행 시 서비스는 변경된 파드2의 IP를 알게 되며, 파드1의 통신 요청을 변경된 파드2의 IP 인 172.16.1.3 으로 전달하게 됩니다.

서비스는 부하분산 장비 <Layer 4 Switch> 와 유사하게 요청 트래픽을 **엔드포인트** <Endpoints> 에 전달합니다. 엔드포인트는 서비스에 연동된 파드의 집합입니다.

[그림 6-3] 서비스를 통한 파드 통신



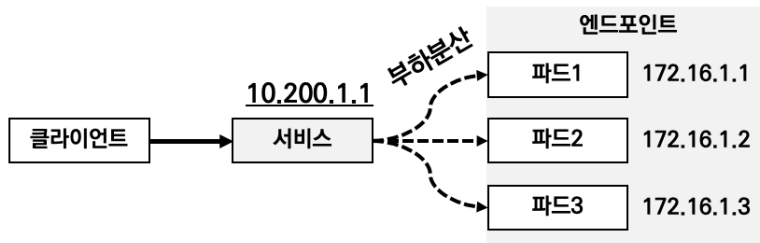
6.1.3 서비스 동작

서비스 동작에 대해서 알아보겠습니다.

첫 번째로, 서비스를 통하여 **부하분산** 접속이 가능합니다.

아래 그림 처럼 클라이언트는 서비스 10.200.1.1 IP 로 반복 접속 시, 파드1 파드2 파드3 으로 부하 분산되어 접속이 됩니다.

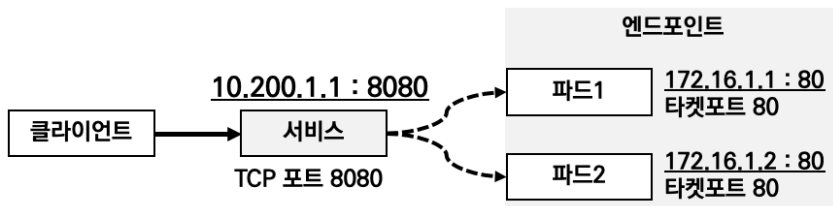
[그림 6-4] 서비스를 통한 부하분산



두 번째로, 서비스 리소스를 생성 시 **포트** <port> 와 **타겟포트** <targetPort> 와 **레이블** <label> 과 **셀렉터** <selector> 에 대한 동작과 이해가 필요합니다.

아래 그림 처럼 클라이언트는 서비스의 IP와 포트인 TCP 10.200.1.1:8080 로 접속 하게 됩니다. 이때 접속하는 목적지 포트 8080 을 서비스 리소스에서는 **포트**라고 합니다. 서비스는 해당 접속 요청을 엔드 포인트의 파드의 IP와 타겟포트로 전달하게 됩니다. 예를 들면 파드1의 경우 TCP 172.16.1.1:80 으로 전달하게 되고, 목적지 포트 80 을 서비스 리소스에서는 **타겟포트**라고 합니다.

[그림 6-5] 포트와 타겟포트



서비스에 연동된 파드의 집합은 레이블과 셀렉터로 서비스 리소스에 설정 할 수 있습니다.

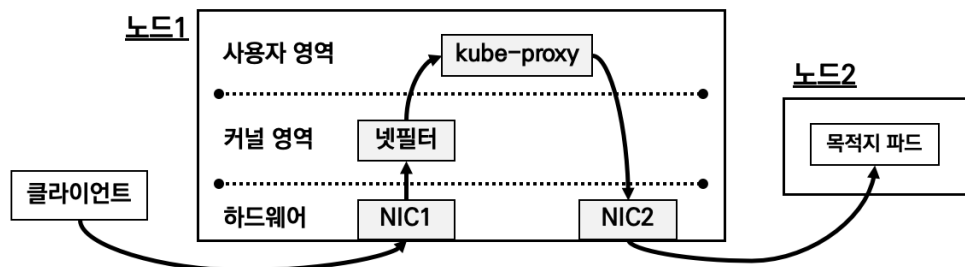
레이블은 말 그대로 관리의 목적으로 알아보기 편하게 메모를 작성하는 것으로 볼 수 있습니다. 예를 들면 특정 파드들에 'app=web' 이라는 레이블을 설정할 수 있습니다.

서비스에 **셀렉터**에 'app=web' 를 설정 시, 해당 서비스는 클러스터에 존재하는 파드 중에서 레이블에 'app=web' 이 설정된 파드들을 엔드포인트로 인식하게 됩니다. 만약, 신규 파드가 생성 시 레이블에 'app=web' 이 설정되어 있다면 쿠버네티스 엔드포인트 컨트롤러 <Endpoints Controller> 가 지속적인 모니터링을 통해, 엔드포인트에 해당 파드가 연동이 됩니다.

세 번째로, **kube-proxy 모드** 입니다. kube-proxy 는 서비스 통신 동작에 대한 설정을 관리합니다. kube-proxy 는 데몬셋으로 배포되어, 모든 쿠버네티스 노드에 파드가 생성됩니다. kube-proxy 모드는 **유저 스페이스** <User Space> **proxy 모드**와 **iptables proxy 모드**, 그리고 **IPVS proxy 모드**가 있습니다.

운영체제(OS) 는 커널 <kernel> 과 애플리케이션 <application> 으로 나눌수 있습니다. 커널은 운영체제의 핵심이며, 시스템의 모든 것을 통제합니다. **커널 영역** <kernel space> 은 운영체제 실행을 위해 필요한 메모리 공간입니다. **사용자 영역** <user space> 은 애플리케이션 프로그램이 동작하기 위해 사용되는 메모리 공간입니다.

[그림 6-6] 유저 스페이스 proxy 모드



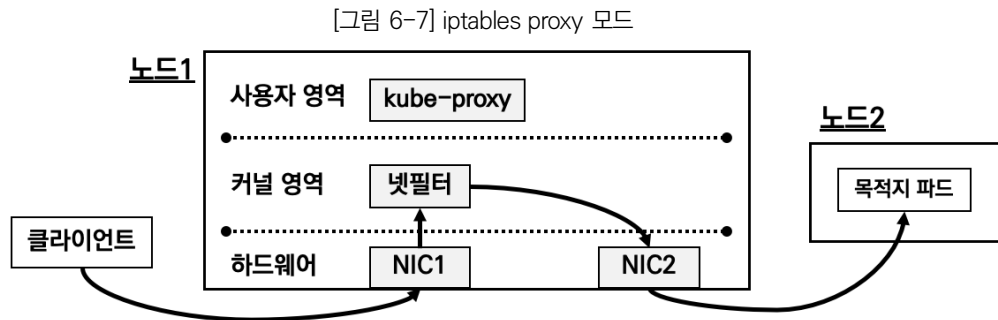
IPTABLES 는 리눅스의 호스트 방화벽/NAT<Network Address Translation> 역할을 합니다.

사용자 영역에서 동작하는 **IPTABLES** 에 정책을 설정 시, 커널 영역에 내장된 **넷필터** <Netfilter> 리눅스 커널 모듈을 통해서 실제로 통제를 수행합니다.

유저 스페이스 proxy 모드 동작에 대해서 알아보겠습니다.

클라이언트의 요청 트래픽이 **노드1**의 **NIC1** 네트워크 인터페이스 하드웨어를 통해 들어옵니다. 커널 영역의 넷필터에 의해서 사용자 영역의 **kube-proxy** 로 전달됩니다. 이후 목적지 정보를 확인하여 **NIC2** 네트워크 인터페이스 하드웨어를 통해 **노드2**의 목적지 파드로 전달됩니다.

유저 스페이스 proxy 모드는 사용자 영역에서 **kube-proxy** 가 동작하고, 커널 영역과 사용자 영역 전환 과정이 필요하여 비효율적인 성능 소모가 발생합니다.



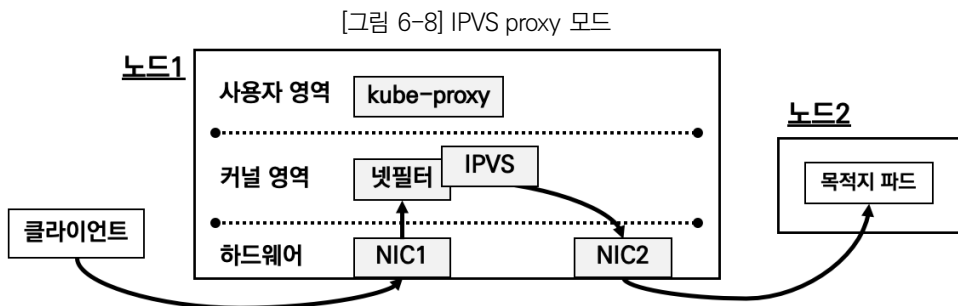
iptables proxy 모드는 쿠버네티스 설치 시 기본 모드입니다.

kube-proxy 통신 트래픽 전달 과정에 직접 관여하지 않습니다. **kube-proxy** 는 서비스 설정 시 관련된 **iptables** 규칙은 넷필터에 적용됩니다. 클라이언트의 요청 트래픽은 커널 영역의 넷필터에 적용되어 있는 규칙에 의해서 **NIC2** 네트워크 인터페이스 하드웨어를 통해 **노드2**의 목적지 파드로 바로 전달 됩니다.

iptables proxy 모드는 트래픽 전달 과정에서 사용자 영역의 **kube-proxy** 를 경유하지 않고, 커널 영역과 사용자 영역 전환이 필요하지 않아서 유저 스페이스 proxy 모드에 비해 시스템 오버헤드 <overhead> 가 줄어듭니다. 오버헤드는 동작 처리를 위해

들어가는 메모리, 처리 시간 등을 말합니다.

다만 iptables proxy 모드는 다양한 부하분산 알고리즘을 제공하지 않고, iptables 규칙이 많아 질 경우 모든 규칙 평가 및 규칙 추가로 인해 지연<Delay> 이 발생합니다. 또한 iptables 규칙이 많은 경우 모든 규칙을 이해하고 운영 관리하기가 용이하지 않습니다.



IPVS proxy 모드는 3가지 모드 중 가장 성능 효율적인 모드 입니다.

IPVS<IP Virtual Server> 는 넷필터에서 동작하는 Layer 4 로드밸런서<Load Balancer> 입니다.

iptables 보다 좀 더 높은 성능 처리를 보여주고, 규칙 갯수를 줄일 수 있습니다.

또한 다양한 부하분산 알고리즘을 제공합니다.

실무 환경에서 가장 권장하는 모드입니다.

이상으로 서비스 동작에 관련된 내용을 알아보았습니다.

6.1.4 서비스 종류

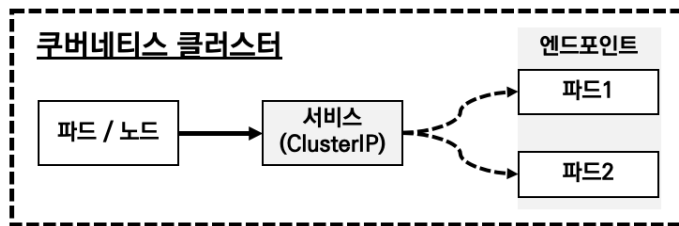
서비스 종류에 대해서 알아보겠습니다.

첫 번째는 **ClusterIP** 서비스입니다.

ClusterIP 서비스는 쿠버네티스 클러스터 내부에서 서비스에 접속 할 수 있습니다.

하지만 클러스터 외부에서는 ClusterIP 서비스로 접속 할 수 없습니다.

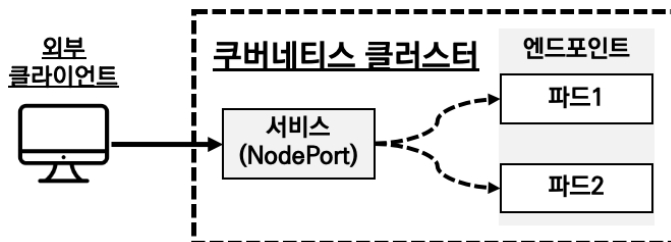
[그림 6-9] ClusterIP 서비스



두 번째는 **NodePort** 서비스입니다.

NodePort 서비스는 쿠버네티스 클러스터 외부에서 노드의 IP:NodePort 로 접속 할 수 있습니다. 이때 노드는 기본적으로 쿠버네티스 클러스터의 모든 노드로 접속 이 가능합니다. 또한 NodePort 서비스는 ClusterIP 서비스를 포함하고 있습니다.

[그림 6-10] NodePort 서비스

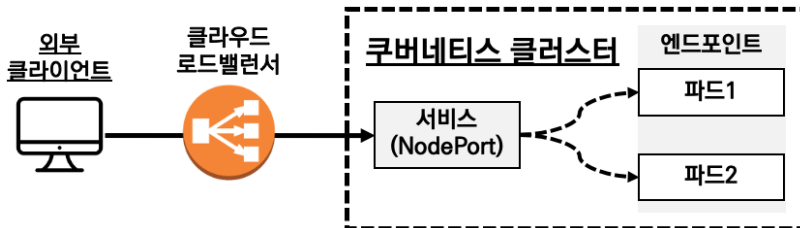


세 번째는 **LoadBalancer** 서비스입니다.

퍼블릭 클라우드 사업자에서 제공하는 로드밸런서 서비스 사용하여 외부에서 쿠버네티스 클러스터로 접속 할 수 있습니다. LoadBalancer 서비스는 NodePort 와 ClusterIP 서비스를 포함하고 있습니다.

참고로 퍼블릭 클라우드 환경이 아닌, 온프레미스 <On-premises> 환경에서도 MetalLB 와 같은 소프트웨어를 이용하여 LoadBalancer 서비스를 사용 할 수 있습니다.

[그림 6-11] LoadBalancer 서비스



네 번째는 **ExternalName** 서비스 입니다.

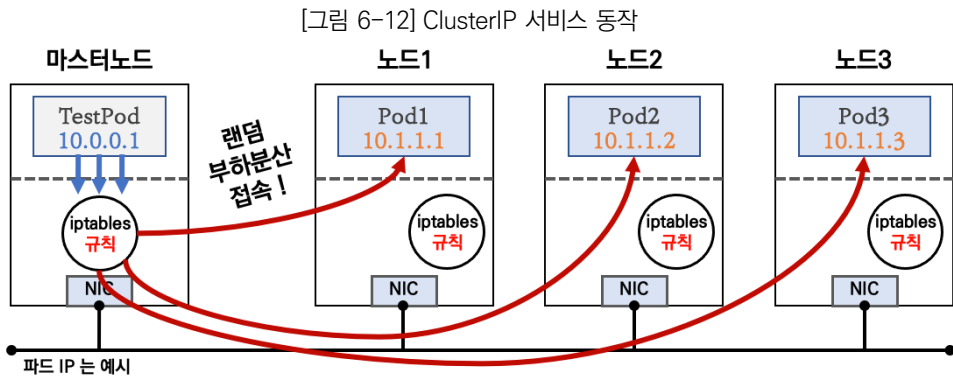
ClusterIP, NodePort, LoadBalancer 서비스의 동작은 클라이언트가 서비스를 통해서 파드로 접속합니다. 하지만 ExternalName 서비스는 쿠버네티스 클러스터 내부에서 서비스의 도메인으로 접속 시 CNAME 레코드를 리턴하여 접속합니다.

ExternalName 서비스는 7장에서 다시 알아보겠습니다.

6.2 ClusterIP 서비스

ClusterIP 서비스 동작을 알아보겠습니다.

ClusterIP 서비스 생성 시 kube-proxy 에 의해서 모든 노드에 iptables 규칙이 설정됩니다. 예를 들면 마스터노드에 배포된 TestPod 파드에서 ClusterIP 서비스로 반복 요청을 하게 되면, 해당 노드에 iptables 규칙에 의해서 서비스에 연동된 파드1, 파드2, 파드3으로 부하분산되어 접속됩니다.



그럼 실습을 통해서 알아보겠습니다.

6.2.1 파드 생성

실습 동작 확인을 위해서 파드를 생성합니다.

위 [그림 6-12] 구성 처럼, 마스터노드에 testpod 파드를 배포하고, 워커노드에 webpod 파드를 배포합니다.

마스터노드의 testpod 는 ClusterIP 서비스에 접속 테스트 용도로 사용합니다.

webpod 파드는 'traefik/whoami' 컨테이너 이미지를 사용합니다. 해당 이미지는 간단한 TCP 80 포트로 웹 서버 동작을 합니다.

마스터 노드에 접속하여 실습을 진행합니다.

터미널

SSH 접속 : 마스터 노드 접속

vagrant ssh k8s-m

kube-proxy 모드 확인 : 아래 모드 mode 에 아무 것도 없을 경우 **iptables** proxy 모드임

예를 들면 **ipvs** proxy 모드의 경우 아래 mode : "**ipvs**" 가 출력됨

root@k8s-m:~# **kubectrl describe cm -n kube-system kube-proxy | grep mode**

mode: ""

마스터노드에 testpod 를 생성을 위한 yaml 파일 확인

root@k8s-m:~# **cat ~/book-k8s-network/6/1-testpod.yaml**

...(생략)...

마스터노드에 testpod 를 생성

root@k8s-m:~# **kubectrl apply -f ~/book-k8s-network/6/1-testpod.yaml**

pod/testpod created

testpod 생성 확인 : 파드의 IP는 실습 환경에 따라 다를 수 있음

root@k8s-m:~# **calicoctl get wep**

WORKLOAD	NODE	NETWORKS	INTERFACE
testpod	k8s-m	172.16.116.5/32	cali47101c87c2f

워커노드에 webpod 를 생성을 위한 yaml 파일 확인

root@k8s-m:~# **cat ~/book-k8s-network/6/2-webpod.yaml**

...(생략)...

워커노드에 webpod 를 생성

root@k8s-m:~# **kubectrl apply -f ~/book-k8s-network/6/2-webpod.yaml**

pod/webpod1 created

pod/webpod2 created

pod/webpod3 created

webpod 생성 확인 : 파드의 IP는 실습 환경에 따라 다를 수 있음

root@k8s-m:~# **calicoctl get wep**

WORKLOAD	NODE	NETWORKS	INTERFACE
testpod	k8s-m	172.16.116.5/32	cali47101c87c2f
webpod1	k8s-w1	172.16.158.1/32	cali778ba511bf9

```

webpod2    k8s-w2    172.16.184.2/32    cali798bceab6f5
webpod3    k8s-w3    172.16.24.3/32    cali10220bc3a76

# webpod 파드의 IP 를 출력
root@k8s-m:~# kubectl get pod -l app=webpod -o jsonpath={.items[*].status.podIP}
172.16.158.1 172.16.184.2 172.16.24.3

# webpod 파드의 IP 를 curl 로 반복 접속
for pod in $(kubectl get pod -l app=webpod -o jsonpath={.items[*].status.podIP}); do curl -s $pod; done
...(생략)...

# webpod 파드의 IP를 변수에 지정
root@k8s-m:~# WEBPOD1=$(kubectl get pod webpod1 -o jsonpath={.status.podIP})
root@k8s-m:~# WEBPOD2=$(kubectl get pod webpod2 -o jsonpath={.status.podIP})
root@k8s-m:~# WEBPOD3=$(kubectl get pod webpod3 -o jsonpath={.status.podIP})

# 마스터노드에서 webpod1 로 curl 접속 : grep 로 출력 결과 필터링 시 해당 파드의 이름 확인
root@k8s-m:~# curl -s $WEBPOD1 | grep Hostname
Hostname: webpod1

# testpod 에서 curl 로 webpod1 접속 : 출력 중 RemoteAddr 은 webpod1 접속 시 클라이언트의 IP와 Port 임
root@k8s-m:~# kubectl exec -it testpod -- curl -s $WEBPOD1 | egrep 'Hostname|RemoteAddr'
Hostname: webpod1
RemoteAddr: 172.16.116.5:40194

```

6.2.2 서비스 생성

이제 ClusterIP 서비스를 생성하고 확인을 해보겠습니다.

```

터미널

# ClusterIP 서비스를 위한 yaml 파일 확인
root@k8s-m:~# cat ~/book-k8s-network/6/3-svc1.yaml

apiVersion: v1
kind: Service
metadata:
  name: svc1 # 서비스 이름

```

```
spec:
  ports:
    - name: webport
      port: 9000 # 서비스 IP 에 접속 시 사용하는 포트 port 를 의미
      targetPort: 80 # 타킷 targetPort 는 서비스를 통해서 목적지 파드로 접속 시 해당 파드로 접속하는 포트를 의미
  selector:
    app: webpod # 셀렉터 아래 app:webpod 레이블이 설정되어 있는 파드들은 해당 서비스에 연동됨
  type: ClusterIP # 서비스 타입

# ClusterIP 서비스 생성
root@k8s-m:~# kubectl apply -f ~/book-k8s-network/6/3-svc1.yaml
service/svc1 created

# ClusterIP 서비스 생성 확인 : CLUSTER-IP 와 PORT 확인, 실습 환경에 따라 CLUSTER-IP 는 다를 수 있음
# CLUSTER-IP 는 메모, 뒤에 testpod 파드의 zsh 실행 후 사용 예정
root@k8s-m:~# kubectl get svc svc1
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
svc1 ClusterIP 10.200.1.111 <none> 9000/TCP 45s

# ClusterIP 서비스 상세 확인 : TargetPort 정보 확인, Endpoints 는 서비스에 연동된 파드들의 정보임
root@k8s-m:~# kubectl describe svc svc1 | egrep 'Port|Endpoints'
Port: webport 9000/TCP
TargetPort: 80/TCP
Endpoints: 172.16.158.1:80,172.16.184.2:80,172.16.24.3:80

# 엔드포인트 정보 확인 : ClusterIP 서비스 생성 시 자동으로 엔드포인트가 생성됨을 확인
# 엔드포인트에 IP와 Port 정보는 파드의 레이블에 app:webpod 설정 되어 있는 파드임
root@k8s-m:~# kubectl get endpoints svc1
NAME ENDPOINTS AGE
svc1 172.16.158.1:80,172.16.184.2:80,172.16.24.3:80 4m13s

# ClusterIP 를 변수에 지정
root@k8s-m:~# SVC1=$(kubectl get svc svc1 -o jsonpath={.spec.clusterIP})

# testpod 에서 ClusterIP 접속 : ClusterIP 접속 시, 부하분산 되어 파드로 접속이 됨
root@k8s-m:~# kubectl exec -it testpod -- curl -s $SVC1:9000 | egrep 'Host|Remote'
Hostname: webpod3
RemoteAddr: 172.16.116.5:34028
```

```
Host: 10.200.1.111:9000

# testpod 에서 zsh 쉘 실행
root@k8s-m:~# kubectl exec -it testpod -- zsh
...(생략)...
Welcome to Netshoot! (github.com/nicolaka/netshoot)
testpod>

# ClusterIP 를 변수에 지정 : 자신의 실습 환경에 맞게 ClusterIP 를 직접 입력
testpod> SVC1=10.200.1.111

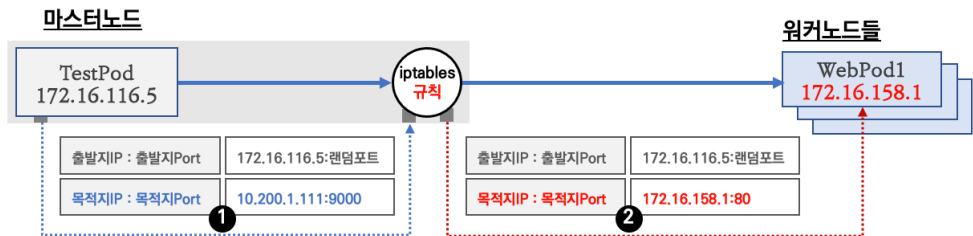
# for 문을 이용하여 SVC1 IP 로 100번 접속을 시도 후 출력되는 내용 중 반복되는 내용의 갯수 출력
# 반복해서 실행을 해보면, SVC1 IP로 curl 접속 시 3개의 파드로 대략 33% 정도로 부하분산 접속됨을 확인
testpod> for i in {1..100}; do curl -s $SVC1:9000 | grep Hostname; done | sort | uniq -c | sort -nr
    36 Hostname: webpod1 # webpod1 로 36번 부하분산 선택되어 접속됨
    34 Hostname: webpod3 # webpod2 로 34번 부하분산 선택되어 접속됨
    30 Hostname: webpod2 # webpod3 로 30번 부하분산 선택되어 접속됨

# testpod 에서 zsh 쉘 빠져나오기
testpod> exit

# SVC1 IP 관련 iptables NAT 테이블 정보 확인
# 서비스 생성 시 kube-proxy 에 의해서 iptables 규칙이 모든 노드에 추가됨
root@k8s-m:~# iptables -t nat -S | grep $SVC1
-A KUBE-SERVICES -d 10.200.1.111/32 -p tcp -m comment --comment "default/svc1:webport cluster IP" -m tcp --dport 9000 -j KUBE-SVC-GQ7VJ6DFEMRF2SE7
-A KUBE-SVC-GQ7VJ6DFEMRF2SE7 ! -s 172.16.0.0/16 -d 10.200.1.111/32 -p tcp -m comment --comment "default/svc1:webport cluster IP" -m tcp --dport 9000 -j KUBE-MARK-MASQ
```

ClusterIP 를 통한 통신 흐름을 알아보겠습니다.

[그림 6-13] ClusterIP 를 통한 통신 흐름



(1) testpod 에서 ClusterIP 주소인 10.200.1.111:9000(TCP) 로 접속을 시도합니다. 이때 출발지 IP는 testpod 의 IP인 172.16.116.5 이고, 출발지 Port 는 랜덤포트가 할당됩니다.

마스터 노드에 **iptables** 의 NAT 테이블에 규칙과 매칭되어 목적지 IP 와 목적지 Port 는 변환이 됩니다. 목적지 IP는 app=webpod 레이블을 가지고 있는 파드 3개 가 대상이 되며, **랜덤 부하분산** 선택이 됩니다. 또한 NAT 가 수행 시 NAT 연결 정보를 기록하고 webpod 를 통해 되돌아온(Return) 트래픽을 확인 후 testpod 로 전달 해줍니다.

참고로 위 동작 처럼 IP가 변환 되는 경우 **NAT**<Network Address Translation> 이라고 하며, Port 가 변환 되는 경우 **PAT**<Port Address Translation> 이라고 합니다.

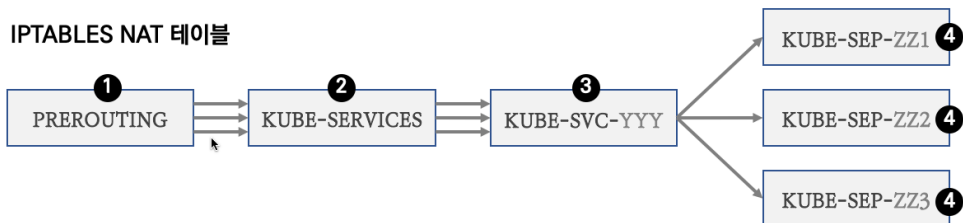
(2) 마스터 노드에 있는 iptables 규칙에 의해 NAT 가 동작하여, 목적지 IP와 목적지 Port 가 변환됩니다.

부하분산 시 webpod1 로 선택 된 경우, 172.16.158.1:80(TCP) 로 요청을 전달합니다. 이때 출발지 IP는 그대로 유지 됩니다.

6.2.3 iptables 규칙

ClusterIP 서비스 생성 시 적용되는 주요 IPTABLES NAT 테이블의 규칙을 알아보겠습니다. iptables 의 다른 테이블(Filter Mangle Raw) 의 규칙은 서비스 동작에 밀접하지는 않아서 별도 설명은 하지 않겠습니다.

[그림 6-14] IPTABLES NAT 테이블



iptables 체인<chain> 은 규칙의 연결 그룹입니다. 규칙에 매칭이 되면 해당 패킷을 어떻게 처리(Target) 할 지 결정하게 됩니다. 일반적으로 허용 혹은 차단하게 되며, 혹은 다른 체인으로 넘길 수 있습니다.

위 그림 처럼, PREROUTING 체인에 매칭되면 KUBE-SERVICES 체인으로 넘기는 처리를 할 수 있습니다.

실습 환경에서 확인해보도록 하겠습니다.

```
터미널

# iptables nat 테이블 확인

root@k8s-m:~# iptables -t nat -S
root@k8s-m:~# iptables -t nat -nvL
...(생략)...
```

(1) PREROUTING 체인의 규칙 확인

```
# 모든 트래픽이 매칭되며 KUBE-SERVICES 로 전달 Jump(점프) 됨
root@k8s-m:~# iptables -t nat -S PREROUTING
-P PREROUTING ACCEPT
-A PREROUTING -m comment --comment "cali:6gwbT8cXdHdC1b1" -j cali-PREROUTING
-A PREROUTING -m comment --comment "kubernetes service portals" -j KUBE-SERVICES
```

(2) KUBE-SERVICES 체인의 규칙 확인

```
# ClusterIP(Port)에 매칭되면 KUBE-SVC-YYY 로 전달됨
# KUBE-SVC-〈YYY〉 에서 〈YYY〉 이름은 실습 환경에 따라 다름
root@k8s-m:~# iptables -t nat -S KUBE-SERVICES
-N KUBE-SERVICES
-A KUBE-SERVICES -d 10.200.1.111/32 -p tcp -m comment --comment "default/svc1:webport cluster IP" -m tcp --dport 9000 -j KUBE-SVC-GQ7VJ6DFEMRF2SE7
...(생략)...
```

(3) KUBE-SVC-YYY 체인의 규칙 확인

```
# SVC-〈YYY〉 규칙은 랜덤 확률(현재 3개의 SEP 가 있으니, 대략 33%)로 KUBE-SEP-ZZZ 로 부하분산 처리됨
# SEP(Service Endpoint) 는 쉽게 말해 파드의 IP와 Port 를 말함, 즉 KUBE-SEP-〈ZZZ〉 로 전달됨
# 아래 KUBE-SEP-〈ZZZ〉 에서 〈ZZZ〉 이름은 실습 환경에 따라 다름
root@k8s-m:~# iptables -t nat -S KUBE-SVC-GQ7VJ6DFEMRF2SE7
```

...(생략)...

```
-A KUBE-SVC-GQ7VJ6DFEMRF2SE7 -m comment --comment "default/svc1:webport" -m statistic --mode random --probability 0.33333333349 -j KUBE-SEP-LJHSFKRTIUY4RZDJ
-A KUBE-SVC-GQ7VJ6DFEMRF2SE7 -m comment --comment "default/svc1:webport" -m statistic --mode random --probability 0.50000000000 -j KUBE-SEP-SNIEU6WJR45JSB43
-A KUBE-SVC-GQ7VJ6DFEMRF2SE7 -m comment --comment "default/svc1:webport" -j KUBE-SEP-B76ETNTWTG5XSE5J
```

(4) KUBE-SEP-ZZZ 체인의 규칙 확인

해당 체인의 규칙은 webpod1 172.16.158.1:80(TCP) 로 DNAT 처리하여 전달됨을 의미

```
root@k8s-m:~# iptables -t nat -S KUBE-SEP-LJHSFKRTIUY4RZDJ
```

...(생략)...

```
-A KUBE-SEP-LJHSFKRTIUY4RZDJ -p tcp -m comment --comment "default/svc1:webport" -m tcp -j DNAT --to-destination 172.16.158.1:80
```

해당 체인의 규칙은 webpod2 172.16.184.2:80(TCP) 로 DNAT 처리하여 전달됨을 의미

```
root@k8s-m:~# iptables -t nat -S KUBE-SEP-SNIEU6WJR45JSB43
```

...(생략)...

```
-A KUBE-SEP-SNIEU6WJR45JSB43 -p tcp -m comment --comment "default/svc1:webport" -m tcp -j DNAT --to-destination 172.16.184.2:80
```

해당 체인의 규칙은 webpod3 172.16.158.1:80(TCP) 로 DNAT 처리하여 전달됨을 의미

```
root@k8s-m:~# iptables -t nat -S KUBE-SEP-B76ETNTWTG5XSE5J
```

...(생략)...

```
-A KUBE-SEP-B76ETNTWTG5XSE5J -p tcp -m comment --comment "default/svc1:webport" -m tcp -j DNAT --to-destination 172.16.24.3:80
```

IPTABLES 규칙을 확인해보면, ClusterIP 로 접속 시, 랜덤 부하분산 되고 DNAT 처리되어 파드로 접속하게 됩니다.

ClusterIP 의 정체는 바로 노드에 설정되어 있는 IPTABLES 규칙에 의해서 처리가 되고, IPTABLES 규칙은 서비스 리소스를 생성하게 되면, kube-proxy 에 의해서 규칙이 추가됩니다.

또한, IPTABLES 는 마치 Layer 4 장비인 로드밸런서 처럼 부하를 분산하고, NAT 동작을 처리합니다.

6.2.4 셀렉터

svc1 서비스는 셀렉터에 'app=webpod' 설정되어 있습니다. 서비스에 연동된 파드 (엔드포인트)는 파드의 레이블<Label>에 'app=webpod' 설정되어 있어야 합니다.

파드에 레이블을 삭제하거나 다른 파드에 레이블을 추가하게 되면 서비스를 통한 부하분산 접속에서 제거되거나 혹은 신규로 추가가 됩니다.

파드에 레이블을 변경 시, 서비스 동작에 대해서 알아보겠습니다.

```
터미널
# 파드에 레이블 정보 확인
root@k8s-m:~# kubectl get pod --show-labels

NAME      READY STATUS   RESTARTS   AGE   LABELS
testpod    1/1   Running    0         4h54m   app=testpod
webpod1    1/1   Running    0         4h56m   app=webpod
webpod2    1/1   Running    0         4h56m   app=webpod
webpod3    1/1   Running    0         4h56m   app=webpod

# 엔드포인트 정보 확인
root@k8s-m:~# kubectl get endpoints svc1
svc1    172.16.158.1:80,172.16.184.2:80,172.16.24.3:80    5h51m

# webpod3 파드에 레이블 제거 : 레이블은 키Key 와 값Value 으로 구성되며, 값에 '-' 지정 시 레이블 제거됨
root@k8s-m:~# kubectl label pod webpod3 app-
pod/webpod3 unlabeled

# 파드에 레이블 정보 확인 : webpod3 파드에 레이블 제거 확인
root@k8s-m:~# kubectl get pod --show-labels

NAME      READY STATUS   RESTARTS   AGE   LABELS
testpod    1/1   Running    0         6h34m   app=testpod
webpod1    1/1   Running    0         6h37m   app=webpod
webpod2    1/1   Running    0         6h37m   app=webpod
webpod3    1/1   Running    0         6h37m   <none>

# 엔드포인트 정보 확인 : SVC1 서비스에 엔드포인트에 webpod3 제거됨을 확인
root@k8s-m:~# kubectl get endpoints svc1
```

```

NAME      ENDPOINTS          AGE
svc1      172.16.158.1:80,172.16.184.2:80  5h53m

# testpod 에 zsh 쉘 실행
root@k8s-m:~# kubectl exec -it testpod -- zsh
-----

# ClusterIP 를 변수에 지정 : 자신의 실습 환경에 맞게 ClusterIP 를 직접 입력
testpod> SVC1=10.200.1.111

# for 문을 이용하여 SVC1 IP 로 100번 접속을 시도 후 출력되는 내용 중 반복되는 내용의 갯수 출력
# webpod3 은 엔드포인트에 포함되지 않아서, webpod1, webpod2 파드가 부하분산 출력됨
testpod> for i in {1..100}; do curl -s $SVC1:9000 | grep Hostname; done | sort | uniq -c | sort -nr
    52 Hostname: webpod1
    48 Hostname: webpod2

# testpod 에서 zsh 쉘 빠져나오기
testpod> exit
-----

# webpod3 파드에 레이블 추가
root@k8s-m:~# kubectl label pod webpod3 app=webpod

# 파드에 레이블 정보 확인
root@k8s-m:~# kubectl get pod --show-labels
...(생략)...

```

서비스 리소스 생성 시 엔드포인트 리소스는 자동으로 생성됩니다.

엔드포인트는 엔드포인트 컨트롤러가 지속적으로 파드를 모니터링 하여, 엔드포인트의 IP/Port 정보를 업데이트 합니다.

해당 서비스에 연동이 되는 엔드포인트가 되기 위해서는, 서비스에 설정된 선택자의 레이블 값을 파드에 설정이 되어 있으면 됩니다.

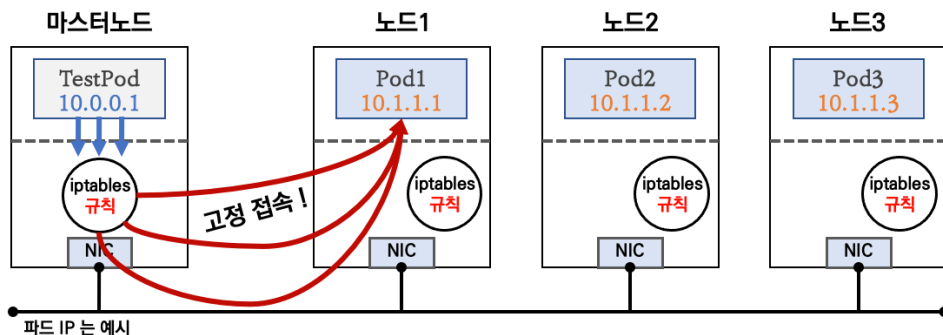
6.2.5 세션어피니티

세션어피니티 <sessionAffinity> 에 대해서 알아보겠습니다.

위에서 알아본 것처럼 서비스에 접속 시 엔드포인트에 파드들로 랜덤 부하분산되어 접속이 됩니다.

만약, 클라이언트의 요청을 매번 동일한 목적지 파드로 전달되기 하기 위해서는 세션어피니티 설정이 필요합니다. 클라이언트가 서비스를 통해서 최초 전달된 파드에 대한 연결 상태를 기록해두고, 이후 동일한 클라이언트가 서비스에 접속 시 연결 상태 정보를 확인하여 최초 전달된 파드, 즉 동일한 혹은 마치 고정적인 파드로 전달 할 수 있습니다.

[그림 6-15] 세션어피니티 설정 시 동작



실습을 통해서 알아보겠습니다.

터미널

```
# svc1 서비스의 세션어피니티 정보 확인 : 기본 설정은 세션어피니티가 비활성화 상태임
root@k8s-m:~# kubectl get svc svc1 -o yaml | grep sessionAffinity
  sessionAffinity: None

# svc1 서비스의 세션어피니티 설정
root@k8s-m:~# kubectl patch svc svc1 -p '{"spec":{"sessionAffinity":"ClientIP"}}'
service/svc1 patched

# svc1 서비스의 세션어피니티 정보 확인 : 세션어피니티가 ClientIP를 기준으로 세션을 기록하는 설정 확인
root@k8s-m:~# kubectl get svc svc1 -o yaml | grep sessionAffinity:
```

```

sessionAffinity: ClientIP

# testpod 에 zsh 쉘 실행
root@k8s-m:~# kubectl exec -it testpod -- zsh
-----

# ClusterIP 를 변수에 지정 : 자신의 실습 환경에 맞게 ClusterIP 를 직접 입력
testpod> SVC1=10.200.1.111

# for 문을 이용하여 SVC1 IP 로 100번 접속을 시도 후 출력되는 내용 중 반복되는 내용의 갯수 출력
# 반복해서 실행해도, 최초 접속한 파드로 고정 접속이 됨을 확인
for i in {1..100}; do curl -s $SVC1:9000 | grep Hostname; done | sort | uniq -c | sort -nr
100 Hostname: webpod2

# testpod 에서 zsh 쉘 빠져나오기
testpod> exit
-----

# iptables 규칙 확인 : 실제로는 iptables 모듈로 동작을 구현
# recent : 동적으로 IP 주소 목록 생성, rcheck : 현재 ip list 에 해당 ip 가 있는지 체크, reap : 오래된 엔트리 삭제
# 세션 기록은 기초 10800초(3시간)이며, 최초 접속 후 3시간 동안 추가 요청이 없으면 세션 정보가 삭제됨
# service.spec.sessionAffinityConfig.clientIP.timeoutSeconds 에 최대 세션 고정 시간 설정 변경 가능함
root@k8s-m:~# iptables -t nat -S | grep recent
-A KUBE-SEP-B76ETNTWTG5XSE5J -p tcp -m comment --comment "default/svc1:webport" -m recent --set --name
KUBE-SEP-B76ETNTWTG5XSE5J --mask 255.255.255.255 --rsource -m tcp -j DNAT --to-destination 172.16.24.3:80
...(생략)...
-A KUBE-SVC-GQ7VJ6DFEMRF2SE7 -m comment --comment "default/svc1:webport" -m recent --rcheck --seconds
10800 --reap --name KUBE-SEP-LJHSFKRTIUY4RZDJ --mask 255.255.255.255 --rsource -j KUBE-SEP-
LJHSFKRTIUY4RZDJ
...(생략)...

# svc1 실습 완료로 svc1 서비스와 파드 삭제
root@k8s-m:~# kubectl delete svc svc1
root@k8s-m:~# kubectl delete pod -l app=webpod
root@k8s-m:~# kubectl delete pod testpod

```

이상으로 ClusterIP 서비스를 알아보았습니다.

쿠버네티스 클러스터 외부에서는 ClusterIP로 직접 통신이 불가능합니다.
다음으로는 클러스터 외부에서 접속 가능한 NodePort 서비스에 대해서 알아보겠습니다.

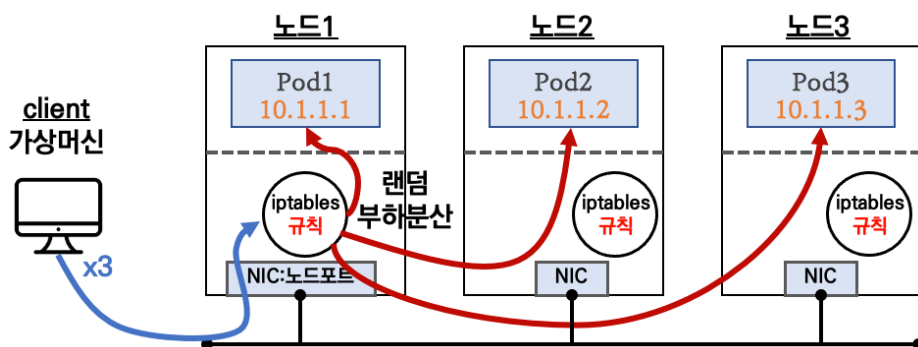
6.3 NodePort 서비스

NodePort 서비스 동작을 알아보겠습니다.

ClusterIP 서비스 생성 시와 동일하게, NodePort 서비스도 생성 시 kube-proxy에 의해서 모든 노드에 iptables 규칙이 설정됩니다.

쿠버네티스 클러스터 외부에 있는 client 가상머신에서 각 노드의 IP:NodePort로 접속할 수 있습니다. 예를 들면 노드1의 IP가 192.168.10.101 이고 NodePort가 30000번 일 경우, client 가상머신은 192.168.10.101:30000(TCP)로 접속할 수 있습니다. 노드1에 진입된 트래픽은 해당 노드에 iptables 규칙에 의해서 서비스에 연동된 파드1, 파드2, 파드3으로 부하분산되어 접속됩니다.

[그림 6-16] NodePort 서비스 동작



6.3.1 파드 생성

실습 동작 확인을 위해서 파드를 생성합니다.

위 [그림 6-16] 구성 처럼, 워커노드에 디플로이먼트 <Deployment> 를 통해 파드를 배포합니다. 파드 이미지는 'k8s.gcr.io/echoserver' 를 사용하며, TCP 8080 포트로 웹 서버 동작을 합니다.

디플로이먼트는 레플리카셋 <ReplicaSet> 의 상위 리소스이며, 애플리케이션의 업데이트와 배포 기능을 제공합니다. 레플리카셋은 정해진 수의 동일한 파드가 실행되도록 동작합니다.

NodePort 서비스 접속 테스트는 쿠버네티스 클러스터 노드에 포함되지 않은 클라이언트 <Client> 가상머신을 통해서 수행합니다.

마스터노드에 접속하여 파드를 생성합니다.

```
터미널
# SSH 접속 : 마스터 노드 접속
vagrant ssh k8s-m

# 디플로이먼트 생성을 위한 yaml 파일 확인 : 'replicas: 3' 지정되어 있어서 파드가 3개가 배포됨
root@k8s-m:~# cat ~/book-k8s-network/6/4-deploy.yaml
...(생략)...

# 디플로이먼트를 생성
root@k8s-m:~# kubectl apply -f ~/book-k8s-network/6/4-deploy.yaml

# 디플로이먼트 생성 확인
root@k8s-m:~# kubectl get deployments
NAME      READY  UP-TO-DATE  AVAILABLE  AGE
deploy-echo  3/3    3           3          3m21s

# 디플로이먼트에 의해서 생성된 레플리카셋 정보 확인
root@k8s-m:~# kubectl get replicaset
NAME          DESIRED  CURRENT  READY  AGE
deploy-echo-d4cc6b75d  3        3        3      9m43s
```

```
# 파드 생성 확인 : 총 3개의 파드 생성 확인
root@k8s-m:~# kubectl get pod

NAME                                READY   STATUS    RESTARTS   AGE
deploy-echo-d4cc6b75d-7d8tg         1/1     Running   0           7m54s
deploy-echo-d4cc6b75d-lwdqc         1/1     Running   0           8m18s
deploy-echo-d4cc6b75d-nmmzp         1/1     Running   0           10m

# 파드의 이름과 IP는 실습 환경에 따라 다를 수 있음
root@k8s-m:~# calicoctl get wep

WORKLOAD          NODE     NETWORKS      INTERFACE
deploy-echo-d4cc6b75d-7d8tg  k8s-w1  172.16.158.1/32  cali0d8a996e595
deploy-echo-d4cc6b75d-lwdqc  k8s-w2  172.16.184.2/32  calif034af2aaa5
deploy-echo-d4cc6b75d-nmmzp  k8s-w3  172.16.24.3/32  cali062a4fd4709

# 파드의 IP를 변수에 지정
root@k8s-m:~# POD1=$(calicoctl get wep | grep k8s-w1 | awk '{print $3}' | cut -d "/" -f 1)
root@k8s-m:~# POD2=$(calicoctl get wep | grep k8s-w2 | awk '{print $3}' | cut -d "/" -f 1)
root@k8s-m:~# POD3=$(calicoctl get wep | grep k8s-w3 | awk '{print $3}' | cut -d "/" -f 1)

# 마스터노드에서 노드1에 배치된 파드1 POD1 웹 접속 확인 : TCP 8080 으로 접속 됨을 확인
# client_address=192.168.10.10 의미는 파드1 웹 파드에 접속 시 클라이언트 IP임
root@k8s-m:~# curl -s $POD1:8080 | egrep 'Hostname|client|host'
Hostname: deploy-echo-d4cc6b75d-7d8tg
      client_address=192.168.10.10
      host=172.16.158.1:8080
```

6.3.2 서비스 생성

NodePort 서비스를 생성하고 확인을 해보겠습니다.

 터미널

```
# NodePort 서비스 생성을 위한 yaml 파일 확인
root@k8s-m:~# cat ~/book-k8s-network/6/5-svc2.yaml

apiVersion: v1
kind: Service
```

```

metadata:
  name: svc2 # 서비스 이름
spec:
  ports:
    - name: webport
      port: 9000 # 서비스 ClusterIP 에 접속 시 사용하는 포트 port 를 의미
      targetPort: 8080 # 타킷 targetPort 는 서비스를 통해서 목적지 파드로 접속 시 해당 파드로 접속하는 포트를 의미
  selector:
    app: deploy-websrv # 셀렉터 아래 app: deploy-websrv 레이블이 설정되어 있는 파드들은 해당 서비스에 연동됨
  type: NodePort # 서비스 타입

# NodePort 서비스 생성
root@k8s-m:~# kubectl apply -f ~/book-k8s-network/6/5-svc2.yaml

# NodePort 서비스 생성 확인 : 실습 환경에 따라 CLUSTER-IP, PORT(S) 는 다를 수 있음
# NodePort 는 ClusterIP를 포함하고 있음, 아래 CLUSTER-IP 와 PORT 에 앞 부분의 9000 포트 번호 확인
# NodePort 정보 중 PORT 에 뒤 부분의 랜덤 할당된 30286 번 포트가 외부에서 접속 가능한 NodePort 임
root@k8s-m:~# kubectl get svc svc2
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
svc2 NodePort 10.200.1.230 <none> 9000:30286/TCP 47s

# NodePort 서비스 상세 확인 : TargetPort 정보 확인, Endpoints 는 서비스에 연동된 파드들의 정보임
root@k8s-m:~# kubectl describe svc svc2 | egrep 'Port|Endpoints'
Type: NodePort
Port: webport 9000/TCP
TargetPort: 8080/TCP
NodePort: webport 30286/TCP # 외부에서 접속 가능한 NodePort 임
Endpoints: 172.16.158.1:8080,172.16.184.2:8080,172.16.24.3:8080

# 엔드포인트 정보 확인 : NodePort 서비스는 ClusterIP를 포함하고, 자동으로 엔드포인트가 생성됨
# 엔드포인트에 IP와 Port 정보는 파드의 레이블에 app=deploy-websrv 설정 되어 있는 파드임
root@k8s-m:~# kubectl get endpoints svc2
NAME ENDPOINTS AGE
svc1 172.16.158.1:80,172.16.184.2:80,172.16.24.3:80 4m13s

# ClusterIP 를 변수에 지정 : NodePort 는 ClusterIP를 포함하여 클러스터 내부에서는 ClusterIP로 접속 가능
root@k8s-m:~# SVC2=$(kubectl get svc svc2 -o jsonpath={.spec.clusterIP})

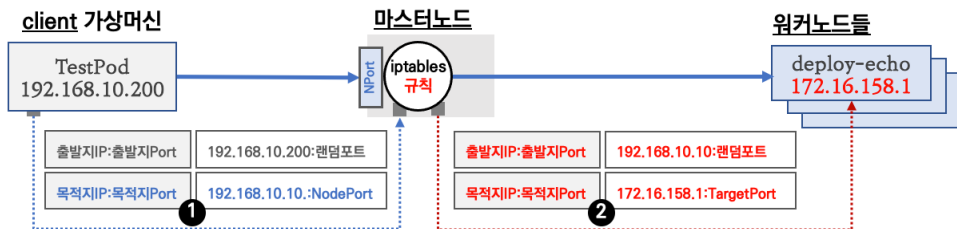
```

```
# 마스터노드에서 ClusterIP 접속 : ClusterIP 접속 시, 부하분산 되어 파드로 접속이 됨
root@k8s-m:~# curl -s $SVC2:9000 | grep Host
Hostname: deploy-echo-d4cc6b75d-7d8tg

# SVC2 IP 관련 iptables NAT 테이블 정보 확인 : 상세 정보 확인은 뒤에서 설명
root@k8s-m:~# iptables -t nat -S
...(생략)...
```

NodePort 서비스 접속을 통한 통신 흐름을 알아보겠습니다.

[그림 6-17] NodePort 를 통한 통신 흐름



(1) client 가상머신에서 마스터노드에 NodePort로 접속을 시도합니다. 물론 워커노드에 NodePort 접속도 가능하지만, 실습 동작 확인에 편의를 위해 마스터노드에서 접속 후 테스트 진행 및 설명을 진행하겠습니다.

이때 출발지 IP는 client 가상머신의 IP인 192.168.10.200 이고, 출발지 Port 는 랜덤포트가 할당됩니다. 목적지 IP는 마스터노드의 IP인 192.168.10.10 이고, 목적지 Port 는 NodePort 서비스 생성 시 할당된 30286 (포트 번호는 실습 환경에 따라 다름) 입니다.

(2) 마스터 노드에 **iptables** 의 NAT 테이블에 규칙과 매칭되어 목적지 IP 와 목적지 Port 는 **변환** <Destination NAT> 이 됩니다. 목적지 IP는 app=deploy-websrv 레이블을 가지고 있는 파드 3개가 대상이 되며, **랜덤 부하분산** 선택이 됩니다.

현재 실습 환경에서는 마스터 노드에는 목적지 파드가 배포되어 있지 않습니다. 즉, 마스터노드는 파드가 배포된 워커노드에 전달을 하게 됩니다. 이때 **출발지 IP**는 마

스터 노드의 IP인 192.168.10.10 로 **변환**<Source NAT> 됩니다.

부하분산시 워커노드1에 배포된 파드로 선택 된 경우, 172.16.158.1:8080 (TargetPort) 로 요청을 전달합니다. 목적지 파드에서 확인되는 출발지 IP는 client 가상머신의 IP가 아니라, 마스터노드의 IP 입니다.

6.3.3 client 가상머신 접속 테스트

실습을 통해서 확인해보도록 하겠습니다. 참고로 client 가상머신에도 kubeconfig 설정이 되어 있어서 kubectl 사용이 가능합니다.

[터미널1] 은 마스터 노드에 SSH 접속하여 명령어를 실행합니다.

[터미널2] 은 client 가상머신에 SSH 접속하여 명령어를 실행합니다.

```
터미널
# [터미널1] SSH 접속 : 마스터 노드 접속
vagrant ssh k8s-m
# [터미널2] SSH 접속 : client 가상머신 접속
vagrant ssh client

# [터미널2] NodePort 서비스 확인
root@client:~# kubectl get svc svc2
NAME    TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
svc2    NodePort    10.200.1.230   <none>         9000:30286/TCP   75m

# [터미널2] client에서 ClusterIP 접속 시도 : 클러스터 외부에서는 ClusterIP 로 접속 불가
root@client:~# SVC2=$(kubectl get svc svc2 -o jsonpath={.spec.clusterIP})
root@client:~# curl -s --connect-timeout 1 $SVC2:9000 -v
* Trying 10.200.1.230:9000...
* After 1000ms connect time, move on!
* connect to 10.200.1.230 port 9000 failed: Connection timed out
...(생략)...

# [터미널2] NodePort 변수에 지정
root@client:~# NPORT=$(kubectl get service svc2 -o jsonpath='{.spec.ports[0].nodePort}')
```

```

# [터미널2] client에서 마스터:NodePort 로 접속 시도
# 쿠버네티스 외부에서 NodePort 를 통하여 목적지 파드로 접속 확인
root@client:~# curl -s 192.168.10.10:$NPORT | egrep 'Host|client|host'
Hostname: deploy-echo-d4cc6b75d-7d8tg # 목적지 파드 이름
      client_address=192.168.10.10 # 목적지 파드에 인입되는 출발지 IP, 즉 클라이언트의 IP 주소
      host=192.168.10.10:30286 # client 가 처음 접속을 시도하는 목적지 정보

# [터미널2] for 문을 이용하여 SVC2 IP 로 100번 접속을 시도 : 파드 3개 부하분산 접속 확인
for i in {1..100}; do curl -s 192.168.10.10:$NPORT | grep Host; done | sort | uniq -c | sort -nr
    34 Hostname: deploy-echo-d4cc6b75d-lwdqc
    34 Hostname: deploy-echo-d4cc6b75d-7d8tg
    32 Hostname: deploy-echo-d4cc6b75d-nmmzp

# [터미널2] client에서 워커노드:NodePort 로 접속 시도 : 모든 노드에 NodePort 접속 가능 확인
root@client:~# curl -s 192.168.10.101:$NPORT | grep Host
root@client:~# curl -s 192.168.10.102:$NPORT | grep Host
root@client:~# curl -s 192.168.10.103:$NPORT | grep Host

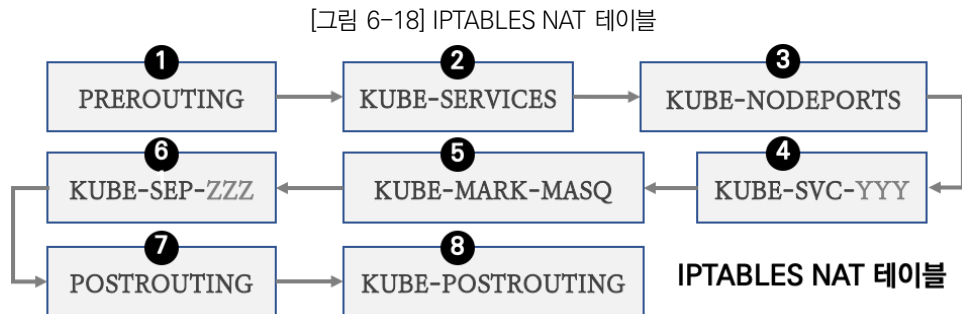
# [터미널1] 마스터노드에서 파드의 실시간 로그 확인 : client 에서 SVC2 로 접속 시도 후 확인
# 파드 이름 뒤에 IP 192.168.10.10 는 해당 파드에 웹 서버에 접속한 웹 클라이언트의 액세스 로그임
# 실제 접속한 client 는 192.168.10.200 이지만, 마스터노드의 IP로 SNAT 변환되었음을 확인
root@k8s-m:~# kubectl -l app=deploy-websrv -f
...(생략)...
[deploy-echo-d4cc6b75d-lwdqc] 192.168.10.10 -- [06/May/2022:08:08:57 +0000] "GET / HTTP/1.1" 200 560 "-"
"curl/7.81.0"
[deploy-echo-d4cc6b75d-nmmzp] 192.168.10.10 -- [06/May/2022:08:08:58 +0000] "GET / HTTP/1.1" 200 560 "-"
"curl/7.81.0"
[deploy-echo-d4cc6b75d-7d8tg] 192.168.10.10 -- [06/May/2022:08:08:58 +0000] "GET / HTTP/1.1" 200 560 "-"
"curl/7.81.0"

# [터미널1] 마스터노드에서 conntrack-tools 로 NAT 변환 정보 확인
root@k8s-m:~# conntrack -L --any-nat | head -n2
conntrack v1.4.6 (conntrack-tools): 9 flow entries have been shown.
tcp      6 114 TIME_WAIT src=192.168.10.200 dst=192.168.10.10 sport=50304 dport=30286 src=172.16.158.1
dst=192.168.10.10 sport=8080 dport=44867 [ASSURED] mark=0 use=1
tcp      6 115 TIME_WAIT src=192.168.10.200 dst=192.168.10.10 sport=50440 dport=30286 src=172.16.184.2
dst=192.168.10.10 sport=8080 dport=44429 [ASSURED] mark=0 use=1
...(생략)...

```

6.3.4 iptables 규칙

NodePort 서비스 생성 시 적용되는 주요 IPTABLES NAT 테이블의 규칙을 알아보겠습니다.



기본 규칙은 ClusterIP 서비스 동작 처리를 위한 규칙과 동일합니다. 차이점은 KUBE-NODEPORTS, KUBE-MARK-MASQ, KUBE-POSTROUTING 체인입니다. 핵심 내용은 NodePort 에 매칭 시 마킹<Mark> 후 출발지 IP를 해당 노드에 있는 네트워크 인터페이스의 IP로 변환<SNAT> 하여 목적지 파드로 전달합니다.

마스터노드에서 확인해보도록 하겠습니다.

```
# 터미널

# iptables nat 테이블 확인
root@k8s-m:~# iptables -t nat -S
root@k8s-m:~# iptables -t nat -nvL
...(생략)...

(1) PREROUTING 체인의 규칙 확인
# 모든 트래픽이 매칭되며 KUBE-SERVICES 로 전달 Jump(점프) 됨
root@k8s-m:~# iptables -t nat -S PREROUTING
-P PREROUTING ACCEPT
-A PREROUTING -m comment --comment "cali:6gwbT8clXdHdC1b1" -j cali-PREROUTING
-A PREROUTING -m comment --comment "kubernetes service portals" -j KUBE-SERVICES
```

(2) KUBE-SERVICES 체인의 규칙 확인

```
# NodePort 가 아닌 ClusterIP(Port)에 접속 시 아래 첫번째 규칙에 매칭되어 KUBE-SVC-YYY로 전달됨
# 해당 노드의 IP를 목적지로 (--dst-type LOCAL) 인입 시 KUBE-NODEPORTS 로 전달됨
root@k8s-m:~# iptables -t nat -S KUBE-SERVICES
...(생략)...
-A KUBE-SERVICES -d 10.200.1.230/32 -p tcp -m comment --comment "default/svc2:webport cluster IP" -m tcp --dport
9000 -j KUBE-SVC-6G65EIPQJPD3EBI3
...(생략)...
-A KUBE-SERVICES -m comment --comment "kubernetes service nodeports; NOTE: this must be the last rule in this chain"
-m addrtype --dst-type LOCAL -j KUBE-NODEPORTS
```

(3) KUBE-NODEPORTS 체인의 규칙 확인

```
# 포트가 (NodePort) 30286 경우 KUBE-SVC-YYY 로 전달됨
root@k8s-m:~# iptables -t nat -S KUBE-NODEPORTS
-A KUBE-NODEPORTS -p tcp -m comment --comment "default/svc2:webport" -m tcp --dport 30286 -j KUBE-SVC-
6G65EIPQJPD3EBI3
```

(4) KUBE-SVC-YYY 체인의 규칙 확인

```
# 포트가 30286 경우 KUBE-MARK-MASQ 로 전달됨
# SVC-〈YYY〉 규칙은 랜덤 확률(현재 3개의 SEP 가 있으니, 대략 33%)로 KUBE-SEP-ZZZ 로 부하분산 처리됨
root@k8s-m:~# iptables -t nat -S KUBE-SVC-6G65EIPQJPD3EBI3
...(생략)...
-A KUBE-SVC-6G65EIPQJPD3EBI3 -p tcp -m comment --comment "default/svc2:webport" -m tcp --dport 30286 -j
KUBE-MARK-MASQ
-A KUBE-SVC-6G65EIPQJPD3EBI3 -m comment --comment "default/svc2:webport" -m statistic --mode random --
probability 0.33333333349 -j KUBE-SEP-43HV466KLDBP6FHF
-A KUBE-SVC-6G65EIPQJPD3EBI3 -m comment --comment "default/svc2:webport" -m statistic --mode random --
probability 0.50000000000 -j KUBE-SEP-IBQXDYLADY6LXQK
-A KUBE-SVC-6G65EIPQJPD3EBI3 -m comment --comment "default/svc2:webport" -j KUBE-SEP-OLTJCF3K5VEHS3C
```

(5) KUBE-MARK-MASQ 체인의 규칙 확인

```
# 규칙에 0x4000 마크 mark 설정
root@k8s-m:~# iptables -t nat -S KUBE-MARK-MASQ
-A KUBE-MARK-MASQ -j MARK --set-xmark 0x4000/0x4000
```

(6) KUBE-SEP-ZZZ 체인의 규칙 확인

```
# 해당 체인의 규칙은 172.16.158.1:8080(TCP) 로 DNAT 처리하여 전달됨을 의미
# 나머지 2개의 KUBE-SEP-ZZZ 도 동작은 ClusterIP 에서 확인과 동일함
```



```
root@k8s-m:~# iptables -t nat -S KUBE-SEP-43HV466KLDBP6FHF
...(생략)...
-A KUBE-SEP-43HV466KLDBP6FHF -p tcp -m comment --comment "default/svc2:webport" -m tcp -j DNAT --to-destination 172.16.158.1:8080
```

(7) POSTROUTING 체인의 규칙 확인

KUBE-POSTROUTING 로 전달 확인

```
root@k8s-m:~# iptables -t nat -S POSTROUTING
-P POSTROUTING ACCEPT
-A POSTROUTING -m comment --comment "cali:O3IYWMrLQYEMJtB5" -j cali-POSTROUTING
-A POSTROUTING -m comment --comment "kubernetes postrouting rules" -j KUBE-POSTROUTING
```

(8) KUBE-POSTROUTING 체인의 규칙 확인

맨 아래의 규칙에 의해서 출발지 IP/Port 를 SNAT 마스크레이딩 변환 처리

! --mark 0x4000/0x4000 -j RETURN 규칙은 느낌표! 는 반대의 의미이며, 0x4000 마크가 없으면 RETURN 으로 KUBE-POSTROUTING 체인을 빠져나가게 되어, 그 아래 있는 규칙을 더 이상 적용 되지 않음

client 에서 인입된 트래픽은 0x4000 마크가 있으므로, RETURN 되지 않고 그 아래 있는 규칙 2개를 적용 받음

결국 맨 아래의 규칙에 의해서 client 의 출발지 IP를 마스터노드의 IP로 변환 SNAT 해서 목적지 파드로 전달함

--random-fully 는 출발지 Port 의 충돌방지를 위해서 랜덤한 주소로 변환을 의미

```
root@k8s-m:~# iptables -t nat -S KUBE-POSTROUTING
-N KUBE-POSTROUTING
-A KUBE-POSTROUTING -m mark ! --mark 0x4000/0x4000 -j RETURN
-A KUBE-POSTROUTING -j MARK --set-xmark 0x4000/0x0
-A KUBE-POSTROUTING -m comment --comment "kubernetes service traffic requiring SNAT" -j MASQUERADE --random-fully
```

외부에서 쿠버네티스 NodePort 서비스를 통해서 접속 시에는 출발지 IP가 접속한 노드의 IP 로 변환<SNAT> 되어서 목적지 파드로 접속하게 됩니다.

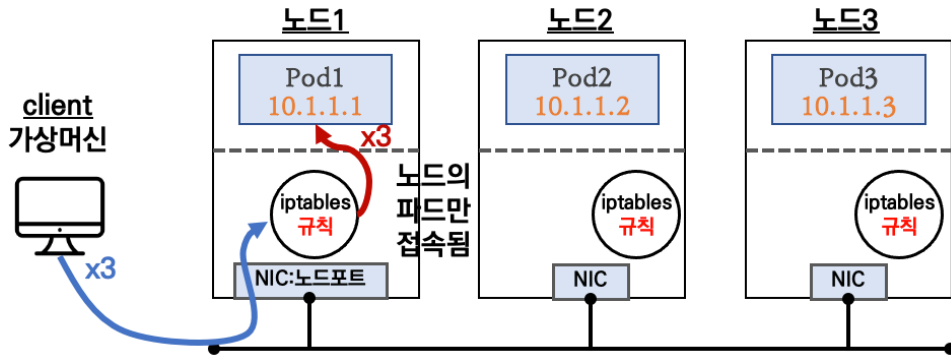
즉, 클라이언트 IP 가 보존되지 않습니다. 법적인 규제나 혹은 다른 이유로 인해 클라이언트 IP 수집이 필요한 경우가 있습니다.

클라이언트 IP 보존을 할 수 있는 설정에 대해서 알아보겠습니다.

6.3.5 externalTrafficPolicy 설정

NodePort 서비스에 'externalTrafficPolicy:Local' 설정 후 동작을 알아보겠습니다.

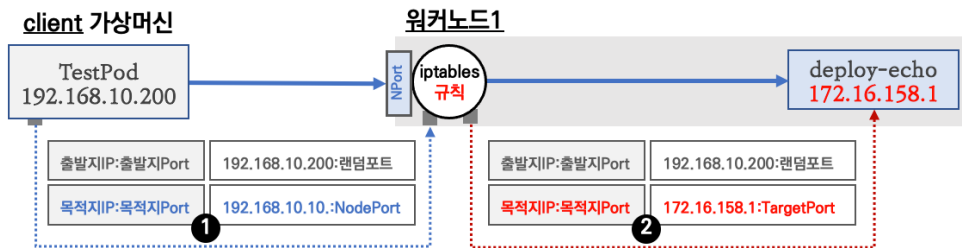
[그림 6-19] NodePort 서비스에 externalTrafficPolicy:Local 설정 시 동작



client 가상머신에서 노드1의 IP에 NodePort 로 접속 시, 노드1에 배포된 파드1로 접속됩니다. 이후 노드1에 NodePort 로 반복 접속을 시도해도, 노드1의 파드1로만 접속이 됩니다. 'externalTrafficPolicy:Local' 설정은 접속한 노드에 배포된 파드로만 전달이 되며, 다른 노드에 배포된 파드로는 전달이 되지 않습니다.

NodePort 에 'externalTrafficPolicy:Local' 설정 시 통신 흐름을 알아보겠습니다.

[그림 6-20] NodePort 서비스에 externalTrafficPolicy:Local 설정 시 통신 흐름



- (1) client 에서 파드가 배포되어 있는 워커노드1에 NodePort 로 접속을 시도합니다.
- (2) 워커노드에 iptables 의 NAT 테이블의 규칙과 매칭되어 목적지 IP와 목적지 Port 는 변환 되지만, 출발지 IP와 Port 는 변환되지 않고 목적지 파드에 전달됩니다. 즉 파드 입장에서 클라이언트 IP가 그대로 인입됩니다.

실습을 통해 알아보겠습니다.

[터미널3] 은 워커 노드1에 SSH 접속하여 명령어를 실행합니다.

[터미널2] 은 client 가상머신에 SSH 접속하여 명령어를 실행합니다.

```
터미널

# [터미널3] SSH 접속 : 워커 노드1 접속
vagrant ssh k8s-w1

# [터미널2] SSH 접속 : client 가상머신 접속
vagrant ssh client

# [터미널3] NodePort 서비스의 External Traffic Policy 정보 확인 : 기본값은 Cluster 임
root@k8s-w1:~# kubectl describe svc svc2 | grep External
External Traffic Policy: Cluster

# [터미널3] NodePort 서비스의 External Traffic Policy:Local 설정
root@k8s-w1:~# kubectl patch svc svc2 -p '{"spec":{"externalTrafficPolicy": "Local"}}'
service/svc2 patched

# [터미널3] NodePort 서비스의 External Traffic Policy 정보 확인 : Local 로 변경 확인
root@k8s-w1:~# kubectl describe svc svc2 | grep External
External Traffic Policy: Local

# [터미널2] NodePort 변수에 지정
root@client:~# NPORT=$(kubectl get service svc2 -o jsonpath='{.spec.ports[0].nodePort}')

# [터미널2] client에서 워커노드1:NodePort 로 접속 시도
# client_address=192.168.10.200 는 파드에서 클라이언트 IP 가 보존(유지) 됨을 확인
root@client:~# curl -s 192.168.10.101:$NPORT | egrep 'Host|client|host'
Hostname: deploy-echo-d4cc6b75d-7d8tg
      client_address=192.168.10.200
      host=192.168.10.101:30286

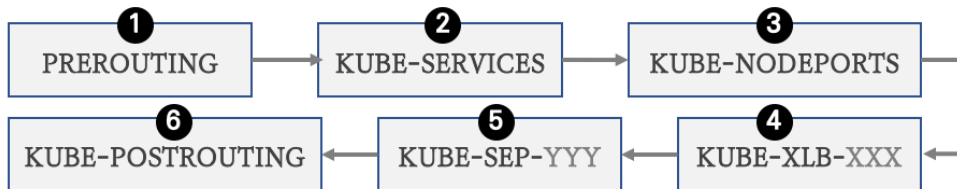
# [터미널2] for 문을 이용하여 SVC2 IP 로 100번 접속을 시도 : 노드1에 배포된 파드로만 접속됨을 확인
for i in {1..100}; do curl -s 192.168.10.101:$NPORT | grep Host; done | sort | uniq -c | sort -nr
100 Hostname: deploy-echo-d4cc6b75d-7d8tg
```

```
# [터미널3] 워커노드1에서 파드의 실시간 로그 확인 : client 접속 시도 시 로그 출력, 클라이언트 IP 유지 확인
root@k8s-w1:~# kubectl logs -l app=deploy-websrv -f
192.168.10.200 -- [06/May/2022:10:19:46 +0000] "GET / HTTP/1.1" 200 563 "-" "curl/7.81.0"
192.168.10.200 -- [06/May/2022:10:19:46 +0000] "GET / HTTP/1.1" 200 563 "-" "curl/7.81.0"
```

NodePort 서비스에 'externalTrafficPolicy:Local' 설정 시 IPTables NAT 테이블 규칙을 알아보겠습니다.

[그림 6-21] IPTables NAT 테이블

IPTables NAT 테이블



KUBE-XLB-XXX 체인 규칙에 서비스 엔드포인트^(SEP)에 해당 노드에 있는 파드 갯수만 설정되어 있습니다. 예를 들면 워커노드1에 서비스에 연동된 파드가 1개가 배포되어 있는 경우, KUBE-XLB-XXX에 SEP 규칙은 1개만 설정되어 있습니다. 즉, 서비스에 연동된 파드이지만 다른 노드에 있는 파드의 경우 서비스 엔드포인트^(SEP) 규칙이 없기 때문에, 해당 파드로 부하분산 되지 않습니다.

또한, KUBE-POSTROUTING에 SNAT 규칙이 되지 않아서, 클라이언트의 IP를 그대로 보존 할 수 있습니다.

워커 노드1에서 확인해보도록 하겠습니다.

```
터미널
(1) PREROUTING 체인의 규칙 확인
# 모든 트래픽이 매칭되며 KUBE-SERVICES 로 전달 Jump(점프) 됨
root@k8s-w1:~# iptables -t nat -S PREROUTING
...(생략)...
```

(2) KUBE-SERVICES 체인의 규칙 확인

해당 노드의 IP를 목적지로 (--dst-type LOCAL) 인입 시 KUBE-NODEPORTS 로 전달됨

```
root@k8s-w1:~# iptables -t nat -S KUBE-SERVICES
```

...(생략)...

(3) KUBE-NODEPORTS 체인의 규칙 확인

목적지 포트 30285 매칭 시 KUBE-XLB-XXX 로 전달

```
root@k8s-w1:~# iptables -t nat -S KUBE-NODEPORTS
```

...(생략)...

```
-A KUBE-NODEPORTS -p tcp -m comment --comment "default/svc2:webport" -m tcp --dport 30286 -j KUBE-XLB-6G65EIPQJPD3EBI3
```

(4) KUBE-XLB-XXX 체인의 규칙 확인

KUBE-SEP-YYY 규칙이 1개만 설정되어 있음

```
root@k8s-w1:~# iptables -t nat -S KUBE-XLB-6G65EIPQJPD3EBI3
```

...(생략)...

```
-A KUBE-XLB-6G65EIPQJPD3EBI3 -m comment --comment "default/svc2:webport" -j KUBE-SEP-43HV466KLDBP6FHF
```

(5) KUBE-SEP-YYY 체인의 규칙 확인

KUBE-SEP-YYY 체인의 규칙 확인 : 해당 노드에 배포된 파드의 IP로 전달

```
root@k8s-w1:~# iptables -t nat -S KUBE-SEP-43HV466KLDBP6FHF
```

...(생략)...

```
-A KUBE-SEP-43HV466KLDBP6FHF -p tcp -m comment --comment "default/svc2:webport" -m tcp -j DNAT --to-destination 172.16.158.1:8080
```

POSTROUTING 체인에서는 KUBE-POSTROUTING 체인으로 전달

(6) KUBE-POSTROUTING 체인의 규칙 확인

0x4000 mark 가 되어 있지 않아서 KUBE-POSTROUTING 체인을 빠져나가서 SNAT 규칙이 적용되지 않음

-j RETURN 의미는 해당 체인의 나머지 규칙은 적용되지 않고 빠져나감

```
root@k8s-w1:~# iptables -t nat -S KUBE-POSTROUTING
```

```
-N KUBE-POSTROUTING
```

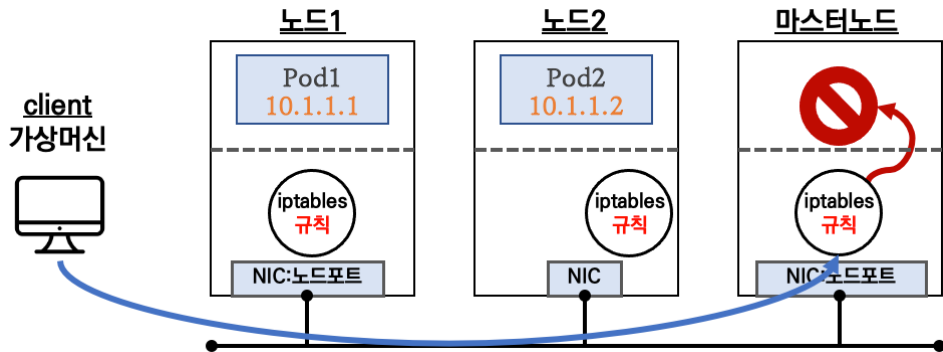
```
-A KUBE-POSTROUTING -m mark ! --mark 0x4000/0x4000 -j RETURN
```

```
-A KUBE-POSTROUTING -j MARK --set-xmark 0x4000/0x0
```

```
-A KUBE-POSTROUTING -m comment --comment "kubernetes service traffic requiring SNAT" -j MASQUERADE --random-fully
```

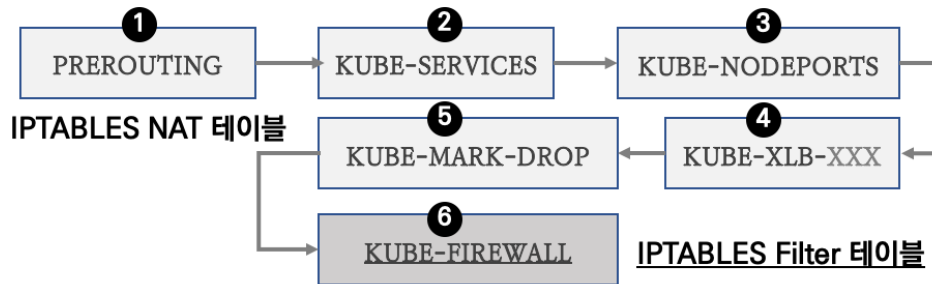
서비스에 연동된 파드가 없는 마스터노드로 접속 시 동작은 어떻게 될까요?

[그림 6-22] 마스터 노드의 NodePort 서비스 접속



마스터노드는 IPTABLES 를 확인하면, KUBE-MARK-DROP 에서 0x8000 mark 되고, Filter 테이블의 KUBE-FIREWALL 체인에 규칙에 의해서 차단^{<DROP>} 됩니다.

[그림 6-23] 마스터 노드의 IPTABLES



마스터노드에서 IPTABLES 를 확인해보겠습니다

```

터미널

(1) PREROUTING 체인의 규칙 확인
# 모든 트래픽이 매칭되며 KUBE-SERVICES 로 전달 Jump(점프) 됨
root@k8s-m:~# iptables -t nat -S PREROUTING
...(생략)...

(2) KUBE-SERVICES 체인의 규칙 확인
# 해당 노드의 IP를 목적지로 (--dst-type LOCAL) 인입 시 KUBE-NODEPORTS 로 전달됨

```

```
root@k8s-m:~# iptables -t nat -S KUBE-SERVICES
```

```
...(생략)...
```

(3) KUBE-NODEPORTS 체인의 규칙 확인

```
# 목적지 포트 30285 매칭 시 KUBE-XLB-XXX 로 전달
```

```
root@k8s-m:~# iptables -t nat -S KUBE-NODEPORTS
```

```
...(생략)...
```

```
-A KUBE-NODEPORTS -p tcp -m comment --comment "default/svc2:webport" -m tcp --dport 30286 -j KUBE-XLB-6G65EIPQJPD3EBI3
```

(4) KUBE-XLB-XXX 체인의 규칙 확인

```
# 마스터노드에 배포된 파드가 없으므로 KUBE-MARK-DROP 로 전달
```

```
root@k8s-m:~# iptables -t nat -S KUBE-XLB-6G65EIPQJPD3EBI3
```

```
...(생략)...
```

```
-A KUBE-XLB-6G65EIPQJPD3EBI3 -m comment --comment "default/svc2:webport has no local endpoints" -j KUBE-MARK-DROP
```

(5) KUBE-MARK-DROP 체인의 규칙 확인

```
# 0x8000 마크 mark 설정됨을 확인
```

```
root@k8s-m:~# iptables -t nat -S KUBE-MARK-DROP
```

```
-A KUBE-MARK-DROP -j MARK --set-xmark 0x8000/0x8000
```

(6) iptables filter 테이블의 KUBE-FIREWALL 체인 규칙 확인

```
# 0x8000 마크 mark 는 차단 DROP 설정됨을 확인
```

```
root@k8s-m:~# iptables -t filter -S KUBE-FIREWALL
```

```
-N KUBE-FIREWALL
```

```
-A KUBE-FIREWALL -m comment --comment "kubernetes firewall for dropping marked packets" -m mark --mark 0x8000/0x8000 -j DROP
```

client 가상머신에서 마스터노드의 NodePort 접속 시도를 해보겠습니다.

 터미널

```
# [터미널2] NodePort 변수에 지정
```

```
root@client:~# NPORT=$(kubectl get service svc2 -o jsonpath='{.spec.ports[0].nodePort}')
```

```
# [터미널2] client에서 마스터:NodePort 로 접속 시도 : 접속 실패 확인
```

```
root@client:~# curl -s --connect-timeout 1 192.168.10.10:$NPORT -v
```

```
* Trying 192.168.10.10:30286...
* After 1000ms connect time, move on!
* connect to 192.168.10.10 port 30286 failed: Connection timed out
```

이상으로 NodePort 에 대해서 알아보았습니다.

외부에서 쿠버네티스 클러스터 내부로 접속을 위해서는 **NodePort** 서비스가 필요합니다. 또한 '**externalTrafficPolicy:Local**' 을 설정하게 되면 외부 클라이언트 IP를 보존하여 수집 할 수 있습니다.

하지만 몇가지 아쉬운점이 있습니다.

첫 번째로, NodePort 로 접속하기 위해서는 **노드의 IP**를 알아야합니다. 즉, 노드의 IP가 외부에 노출되기 때문에 보안상 권장하는 디자인은 아닙니다.

두 번째로, 클라이언트 IP 보존을 위해서 '**externalTrafficPolicy:Local**' 설정 시, **파드가 없는 노드에 접속 시 연결이 실패**하게 되고, 파드가 없는 노드로는 부하분산이 되지 않아서 파드가 있는 노드로의 연결 트래픽이 가증될 수 있습니다.

이를 해결하는 **LoadBalancer** 서비스 타입은 다음 장에서 알아보겠습니다.

실습 완료 후 가상머신을 삭제합니다.

```
터미널
# [터미널1] 실습 완료 후 디플로이먼트, 서비스를 삭제
root@k8s-m:~# kubectl delete deploy deploy-echo
root@k8s-m:~# kubectl delete svc svc2

# [터미널1] 실습 PC 에서 vagrant 로 가상 머신 삭제
exit
vagrant destroy -f
```

끝